

ECE 2011

Introduction to Computer Engineering
Spring 2026

Lecture 20

04-09-2026

Linghao Song

Yale ECE

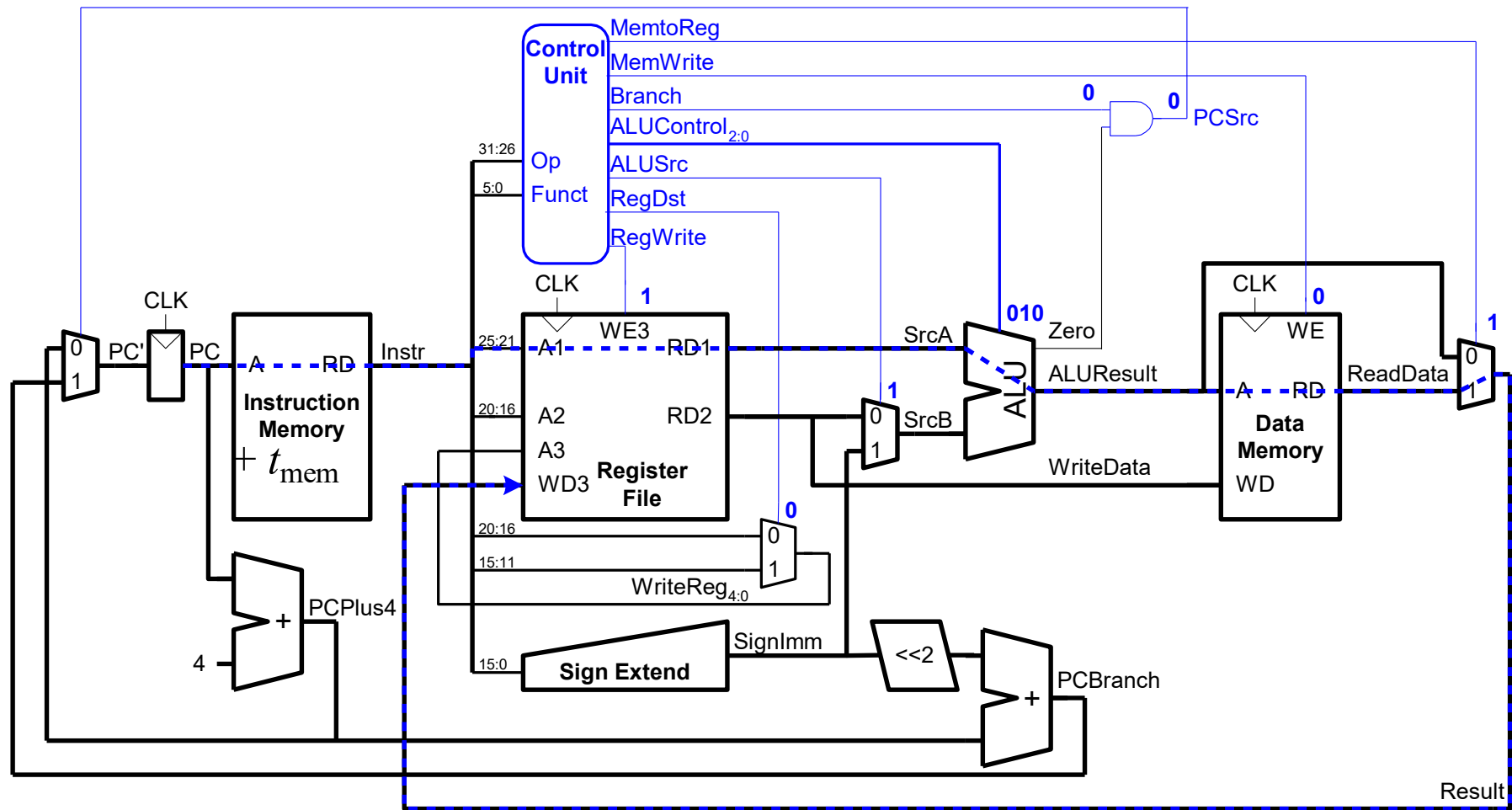
Problem Set 2

- Problem Set 1:
 - April 23 Thursday
 - Select a slot of 75 min at your convenience
 - ALU, MIPS instructions, single-cycle processor, pipeline, memory (Lec 12 - 20)
 - Prepare your cheat sheet: up to 4 pages (not 4 sheets, pieces of papers), scan or take clear photos of your cheat sheet and solutions
 - Submission no later than 11:59pm ET April 23 , submit to Canvas, so you should start working on your pset no later than 10:44pm.
 - No discussion, internet access, textbook, or slides.
 - You may use a non-programmable calculator.

About Assignment 4, Presentation

- In the week of April 14 Tu, 16 Th.
- Your presentation should clearly explain:
 - what problem the paper solves
 - why the problem matters
 - the main idea of the paper
 - how the method or system works at a high level
 - the main results and takeaways
- Focus on clear explanation, not every technical detail.
- 10 – 12 min presentation, plus 3 – 5 min Q&A.
- Tu: Topic 1, 2, and the first paper(HeatViT) of Topic 3
- Th: second paper(ViTCoD) of Topic 3, and Topic 4, 5
- Email me your slides one day before the presentation

Single-Cycle Performance



T_C limited by critical path (1w)

Single-Cycle Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$T_c = ?$$

$$= t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup}$$

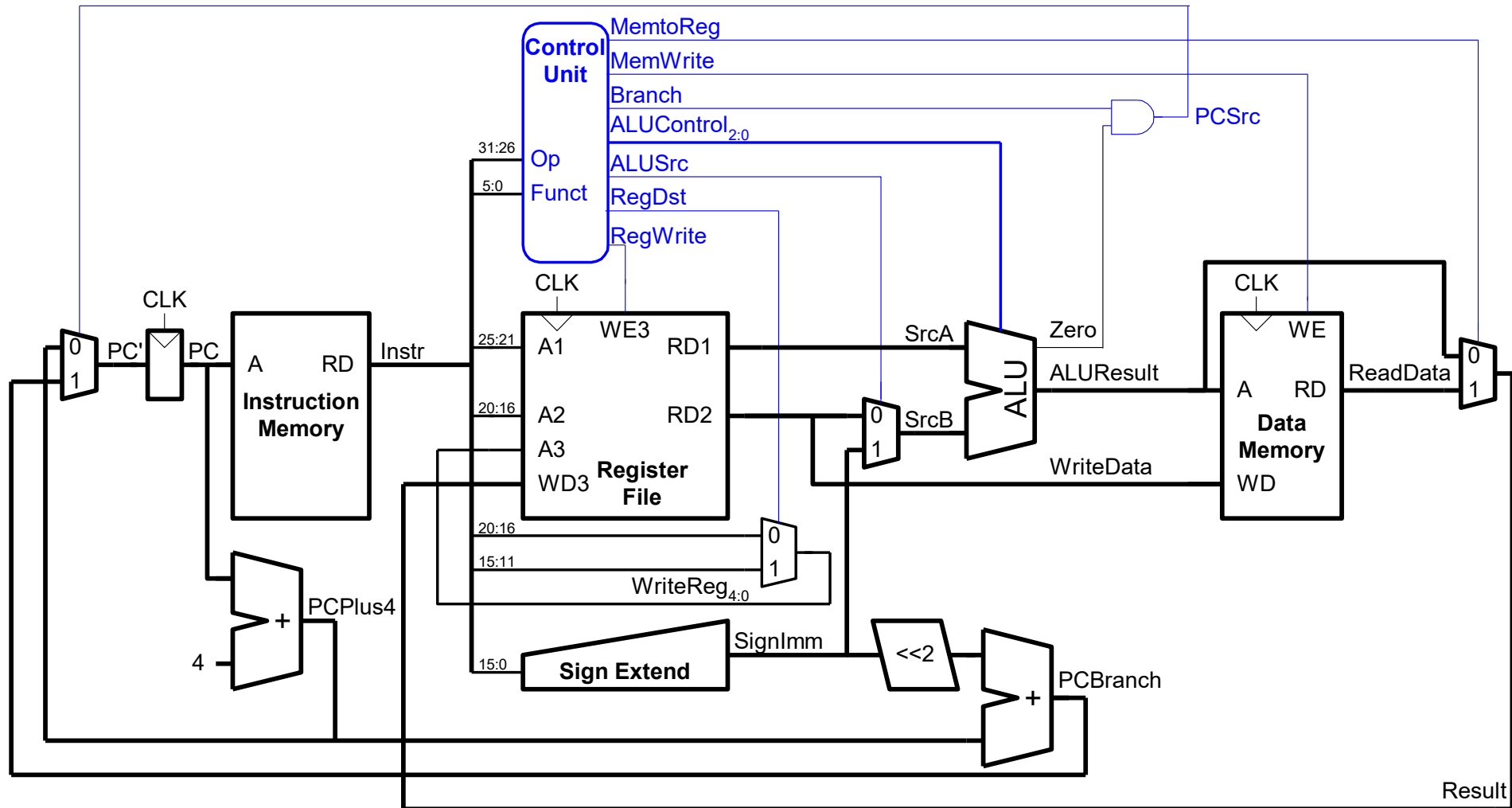
$$= [30 + 2(250) + 150 + 25 + 200 + 20] \text{ ps}$$

$$= 925 \text{ ps}$$

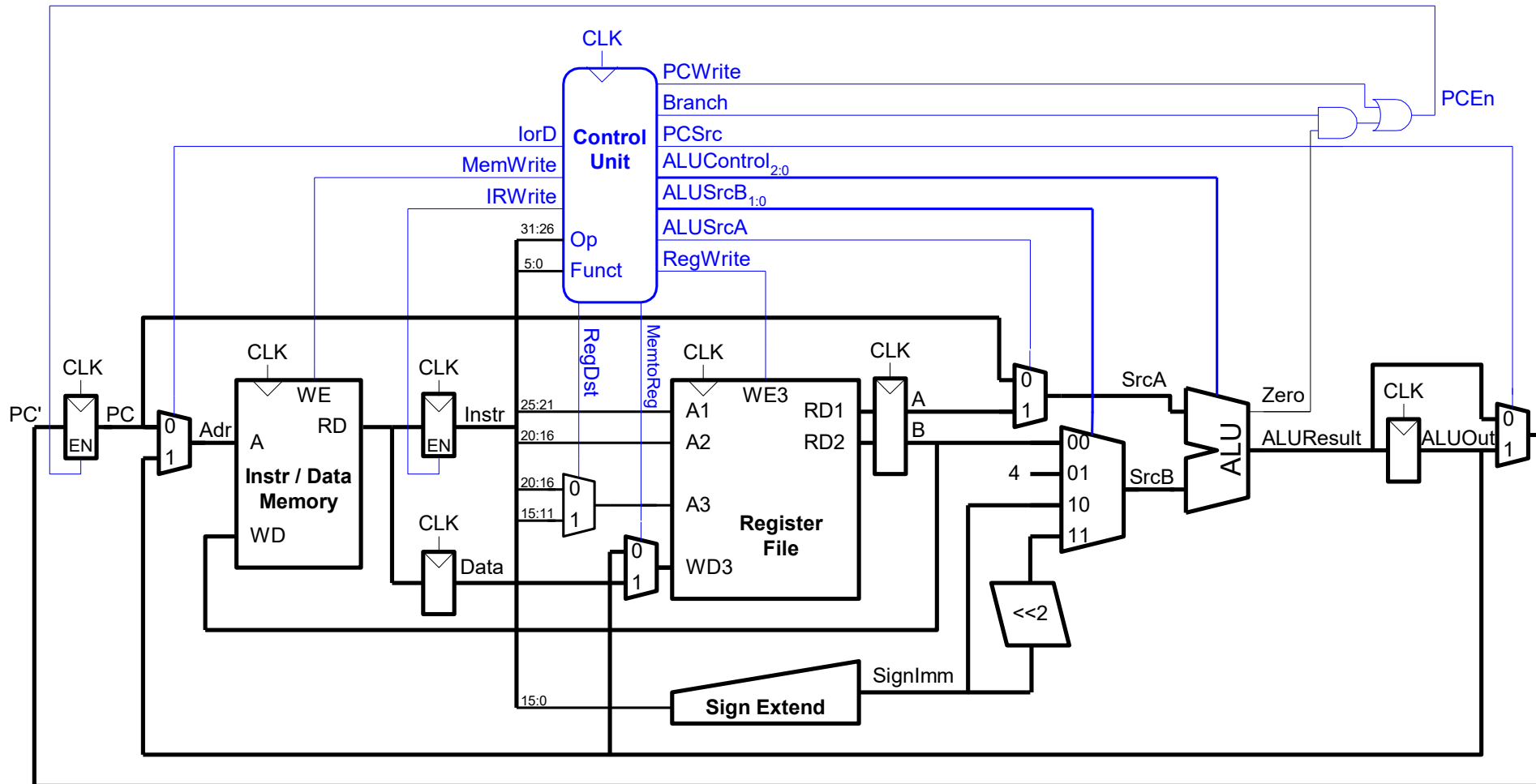
Multicycle MIPS Processor

- **Single-cycle:**
 - + simple
 - cycle time limited by longest instruction (1_w)
 - 2 adders/ALUs & 2 memories
- **Multicycle:**
 - + higher clock speed
 - + simpler instructions run faster
 - + reuse expensive hardware on multiple cycles
 - sequencing overhead paid many times

Single-Cycle Processor



Multicycle Processor

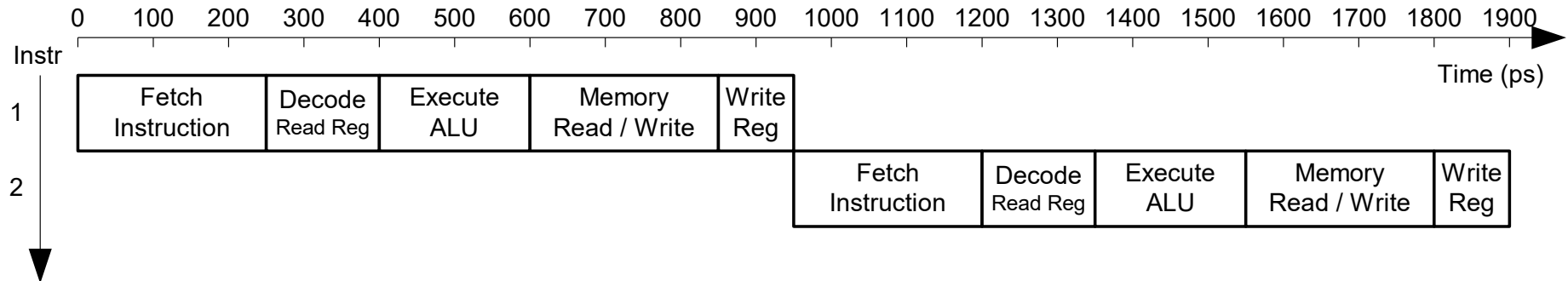


Pipelined MIPS Processor

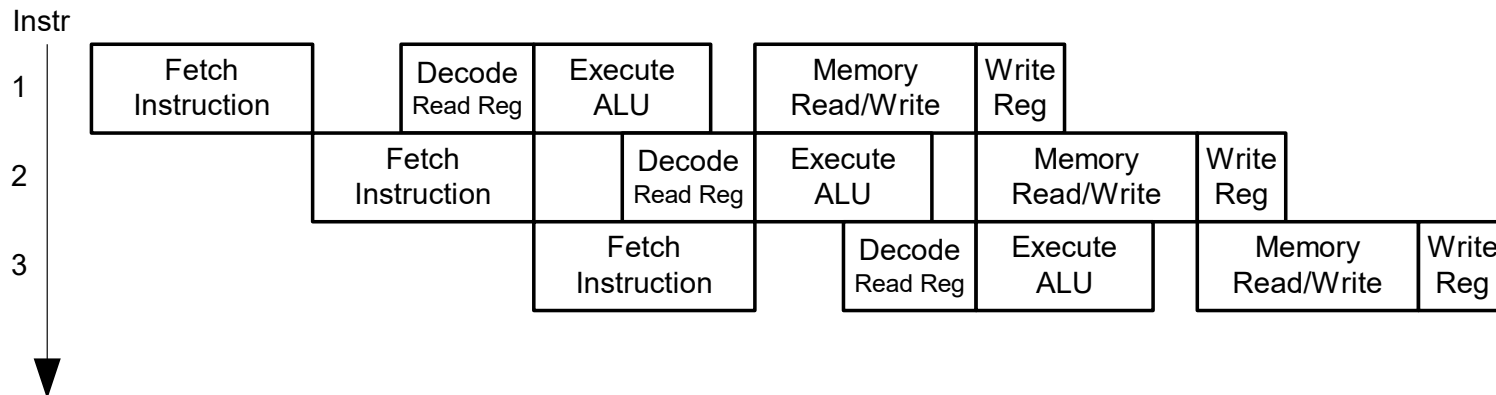
- Temporal parallelism
- Divide single-cycle processor into 5 stages:
 - Fetch
 - Decode
 - Execute
 - Memory
 - Writeback
- Add pipeline registers between stages

Single-Cycle vs. Pipelined

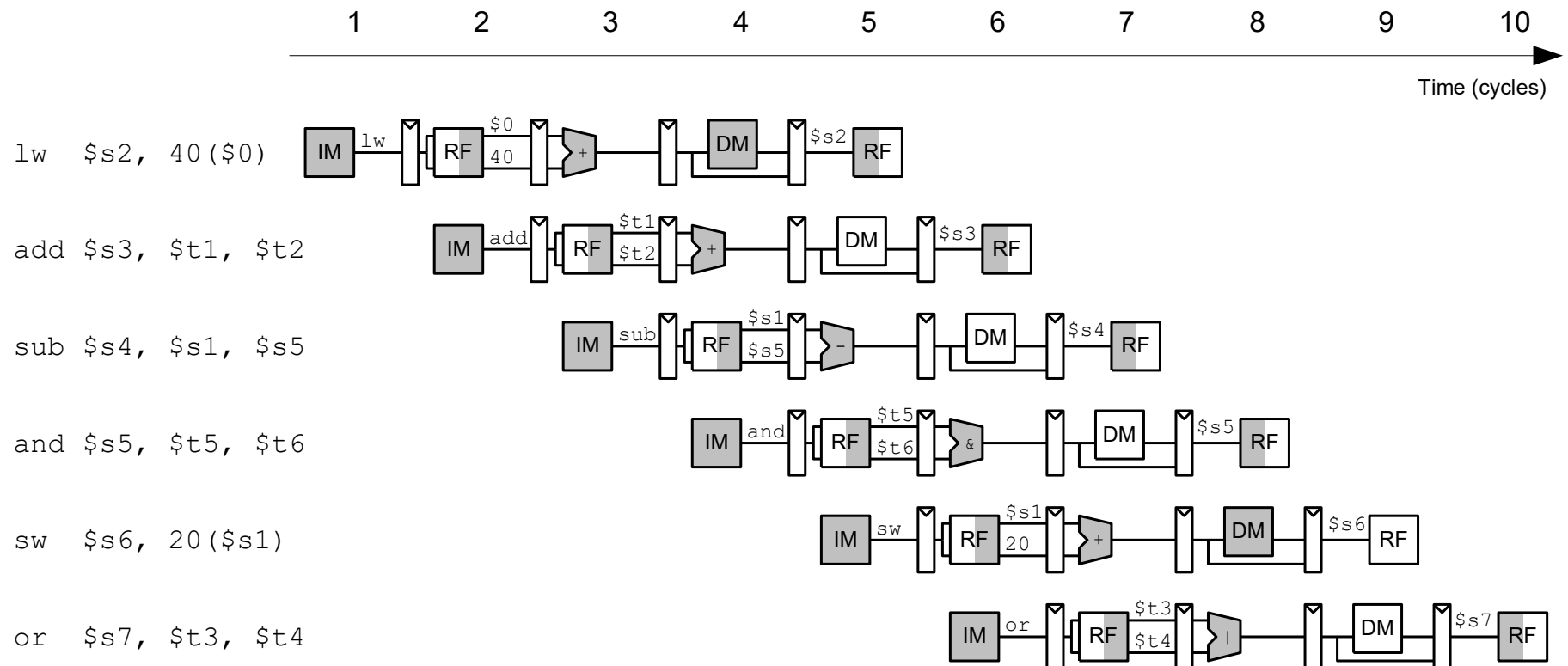
Single-Cycle



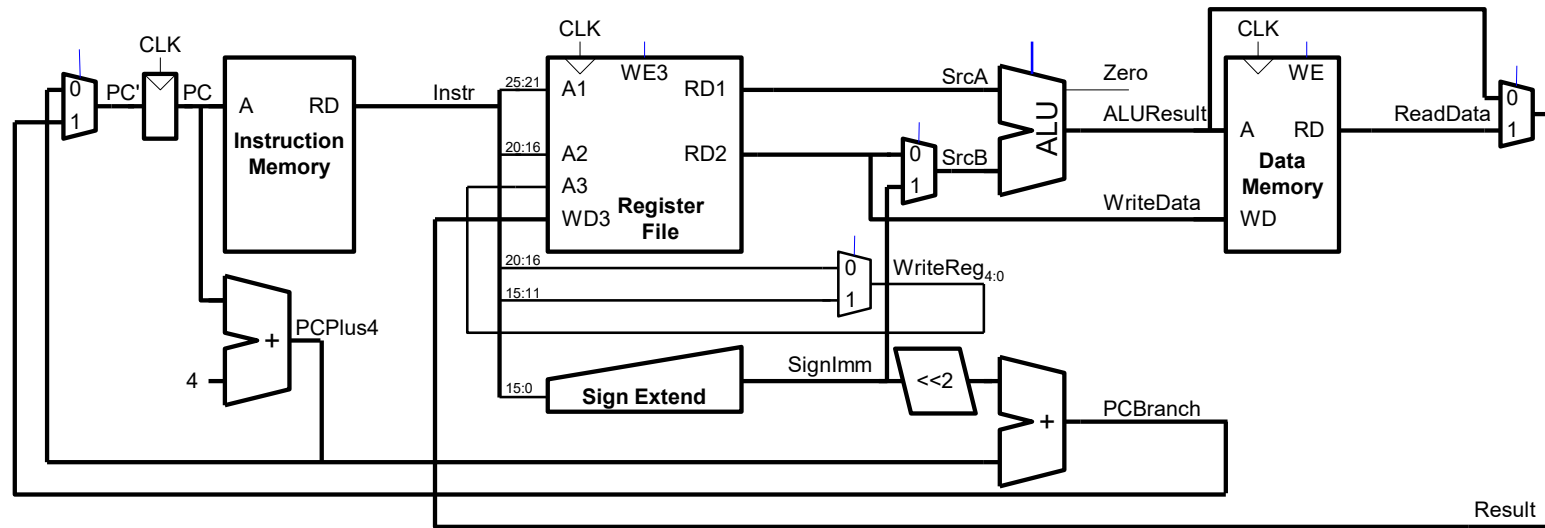
Pipelined



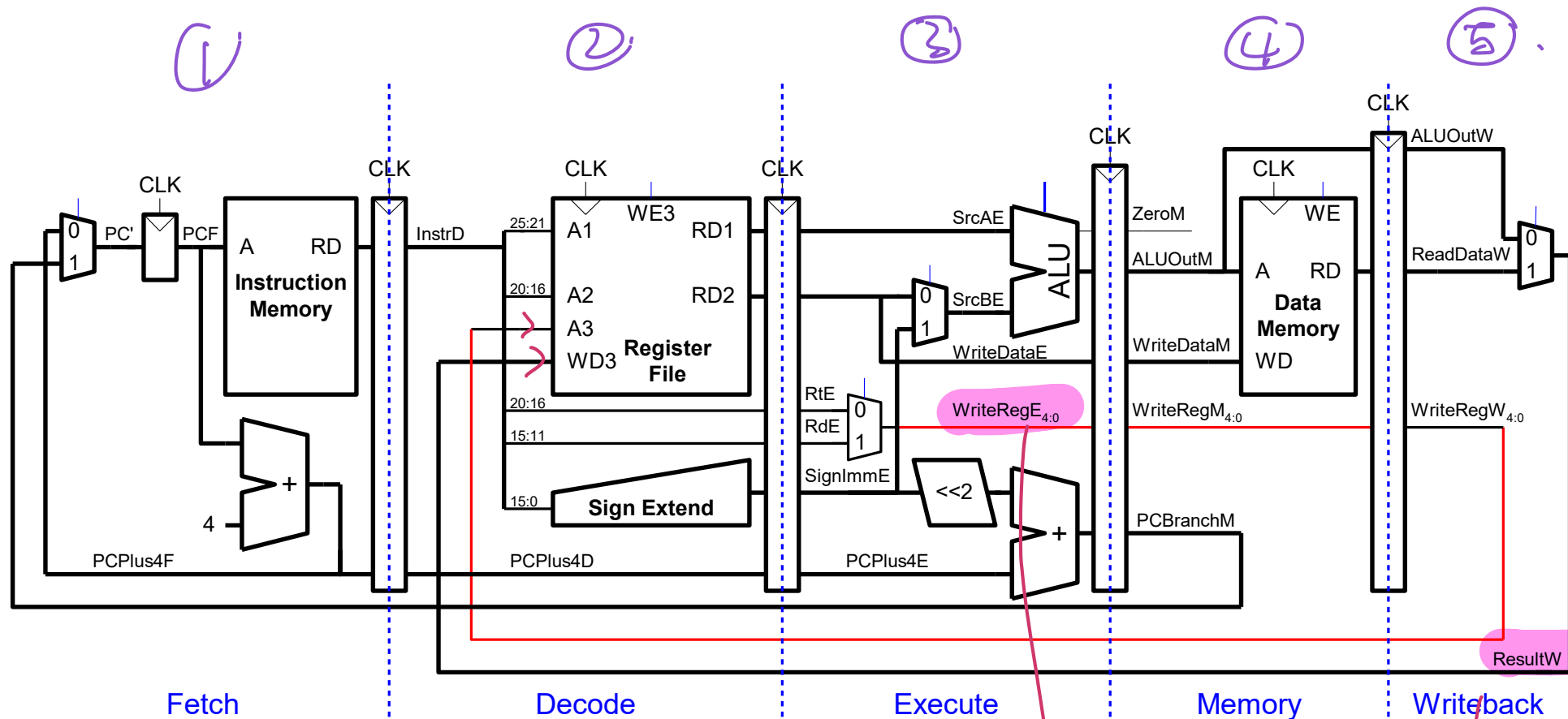
Pipelined Processor Abstraction



Single-Cycle & Pipelined Datapath



Corrected Pipelined Datapath

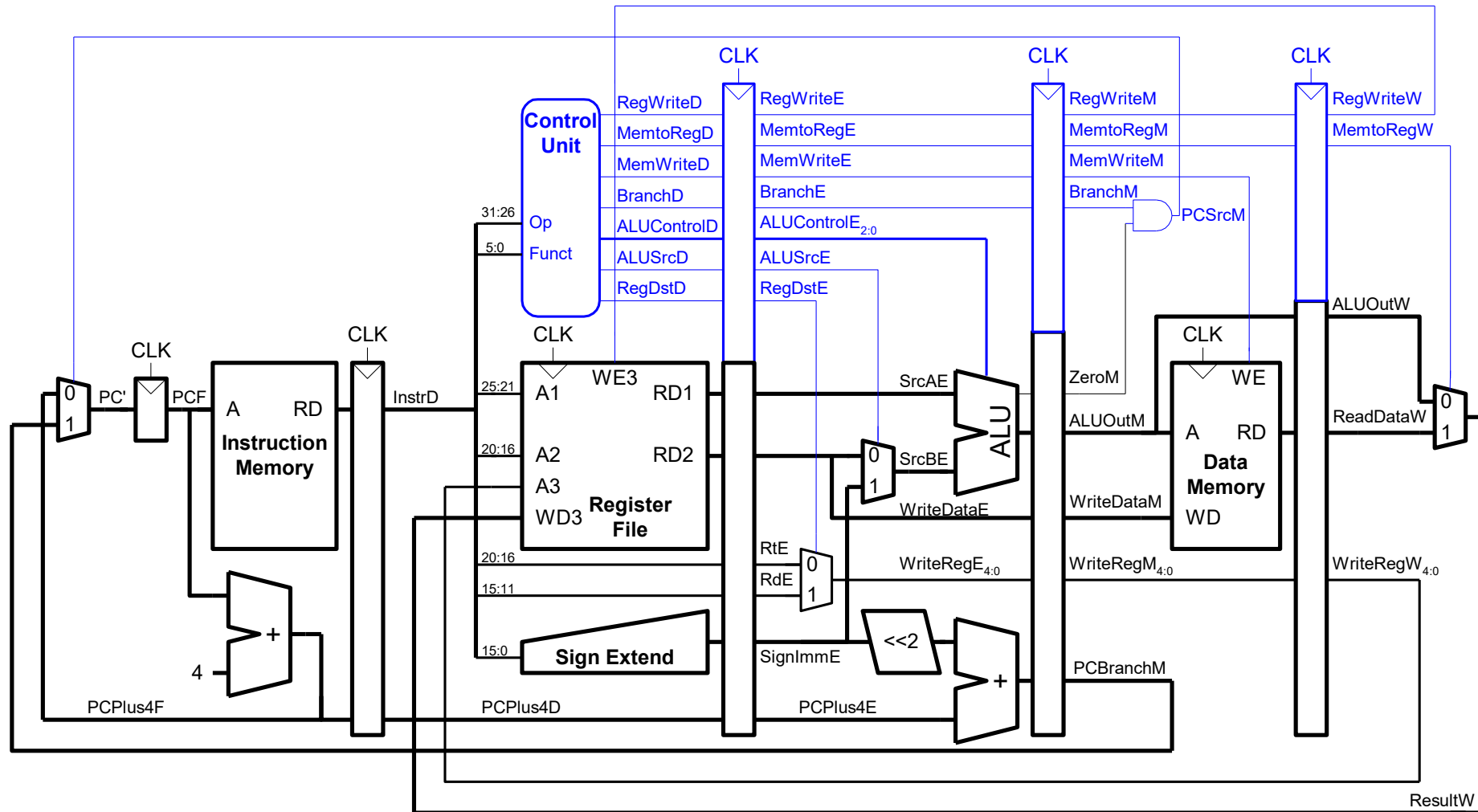


WriteReg must arrive at same time as Result

IDEA: timing > VERY IMPORTANT.

there 2 signals must be ready @ the same time.

Pipelined Processor Control

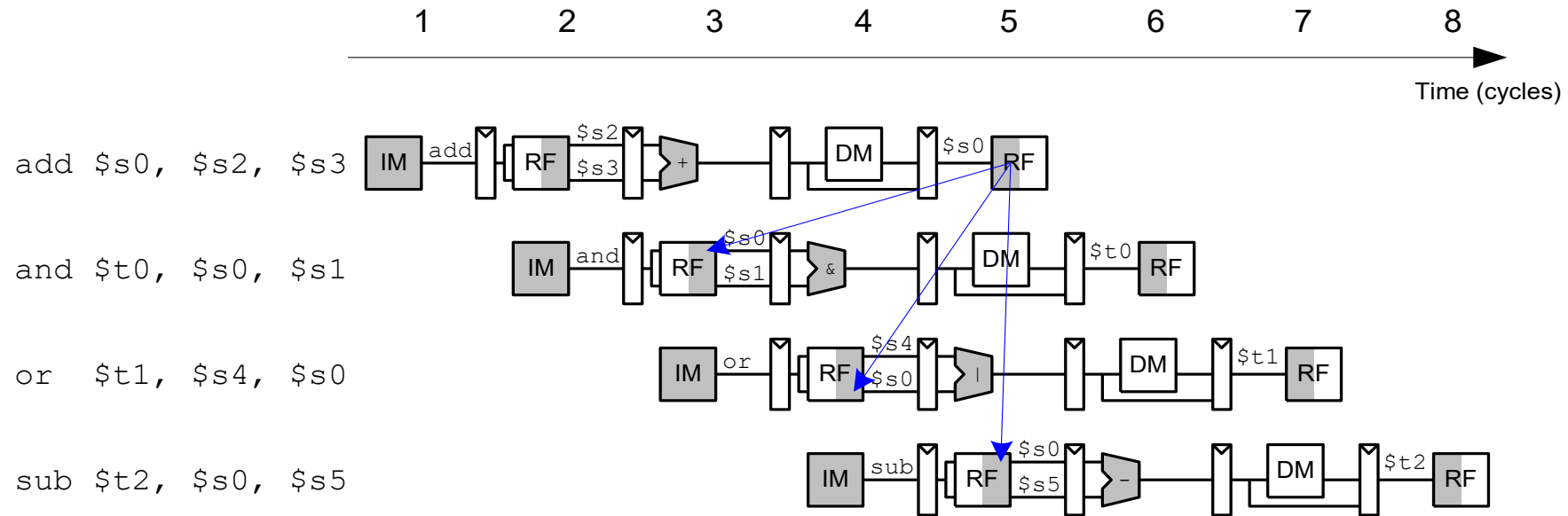


- Same control unit as single-cycle processor
- Control delayed to proper pipeline stage

Pipeline Hazards

- When an instruction depends on result from instruction that hasn't completed
- Types:
 - **Data hazard:** *timing* register value not yet written back to register file
 - **Control hazard:** next instruction not decided yet (caused by branches)

Data Hazard

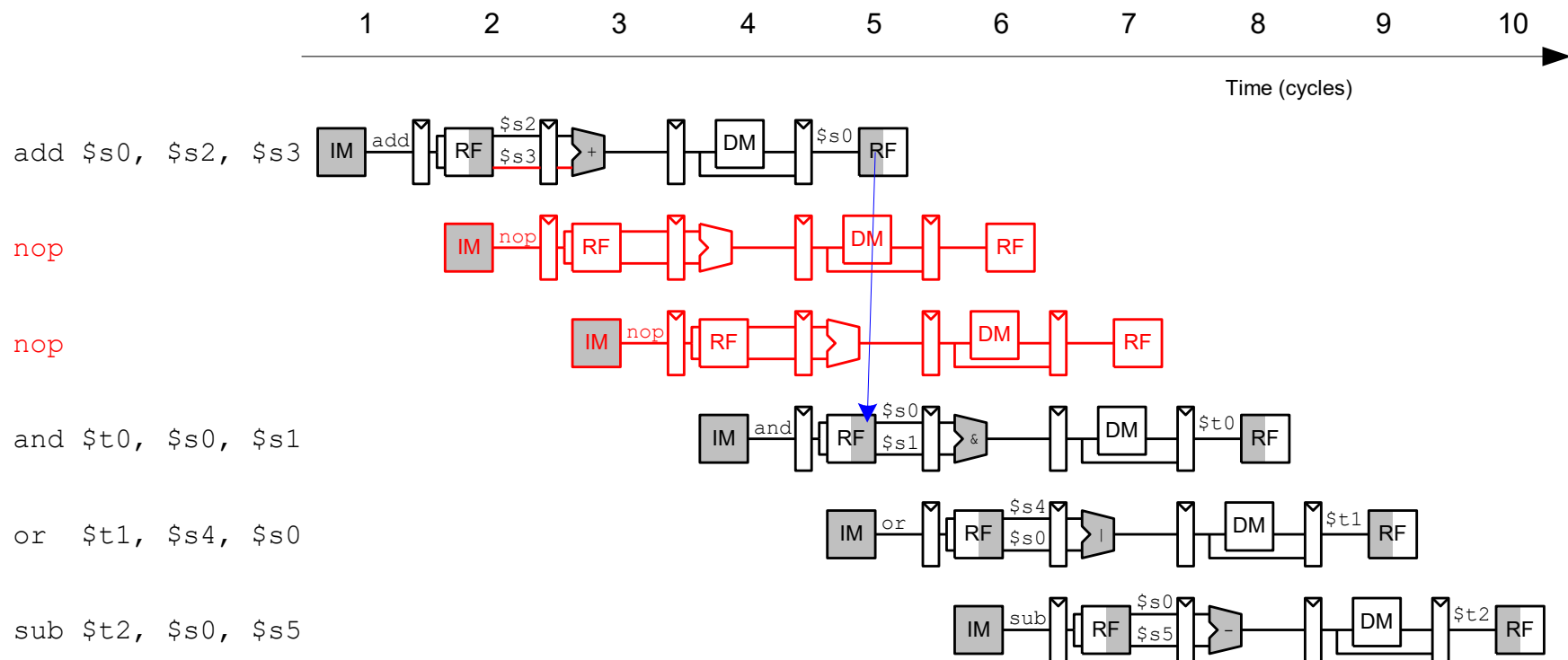


Handling Data Hazards

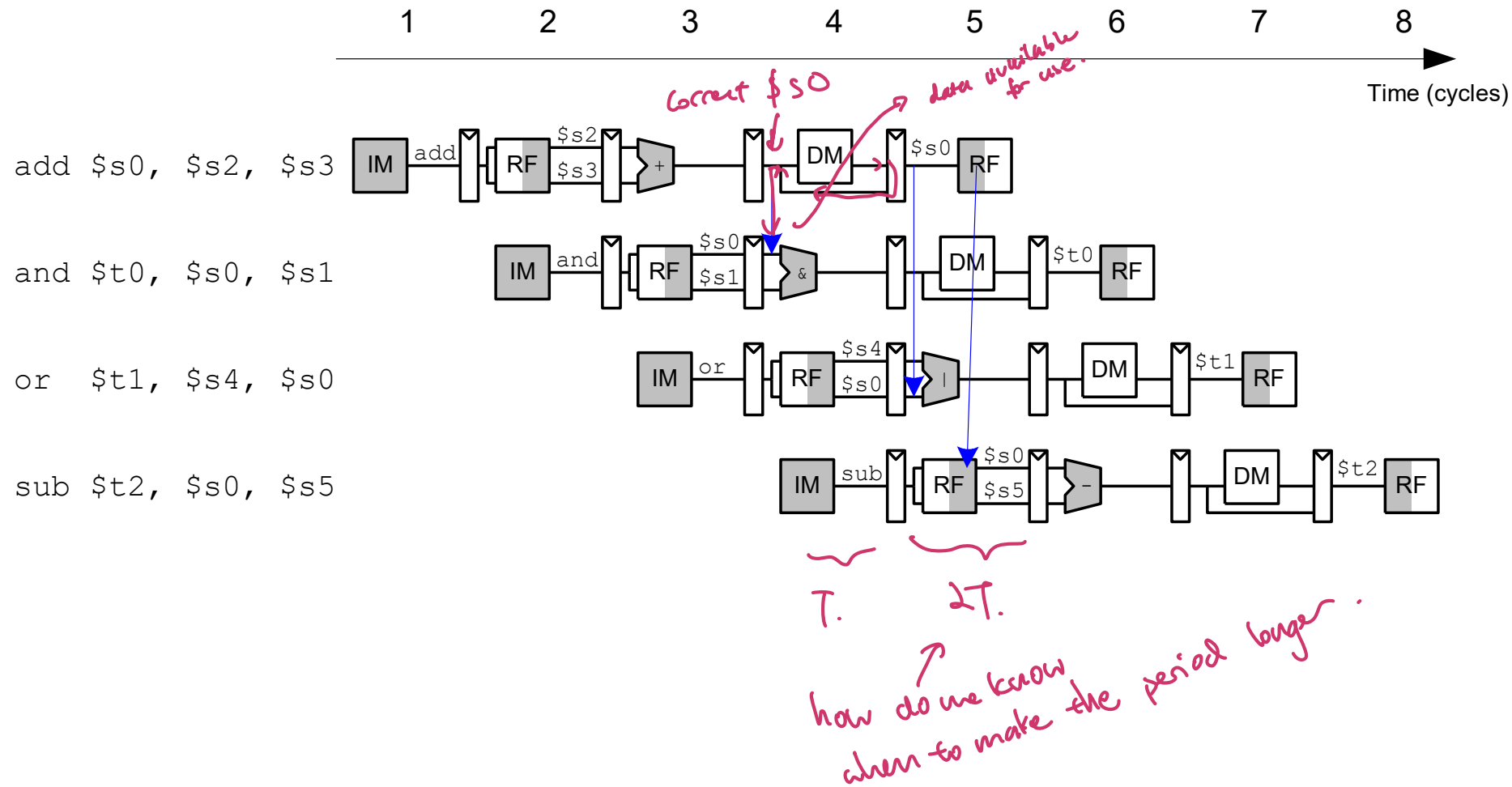
- Insert `nops` in code at compile time
- Rearrange code at compile time
- Forward data at run time
- Stall the processor at run time

Compile-Time Hazard Elimination

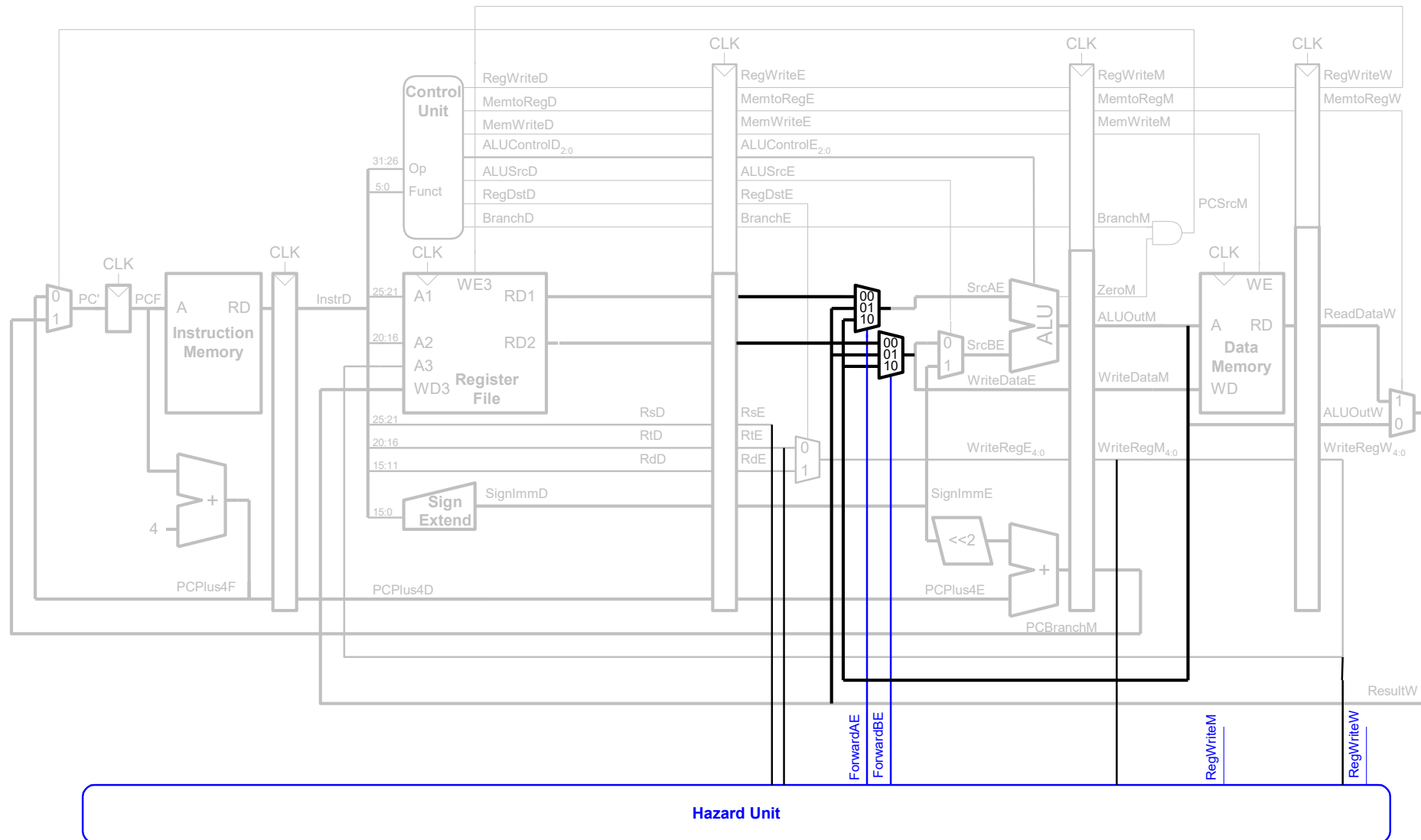
- Insert enough nops for result to be ready
- Or move independent useful instructions forward



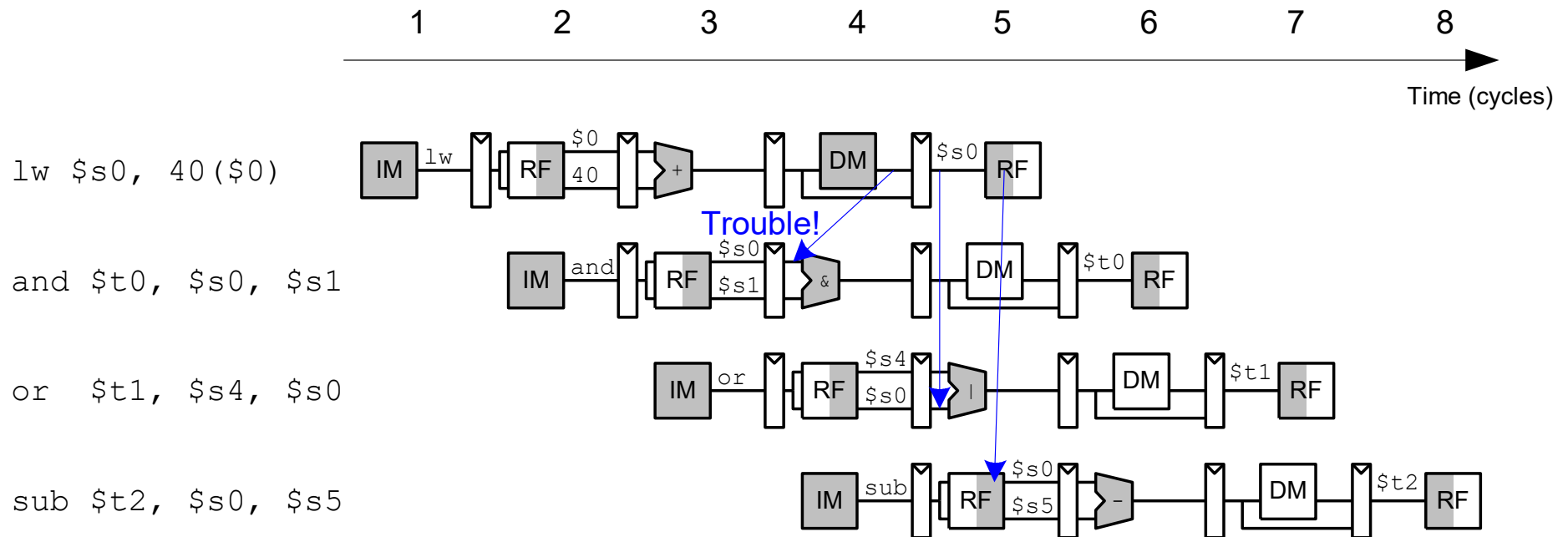
Data Forwarding



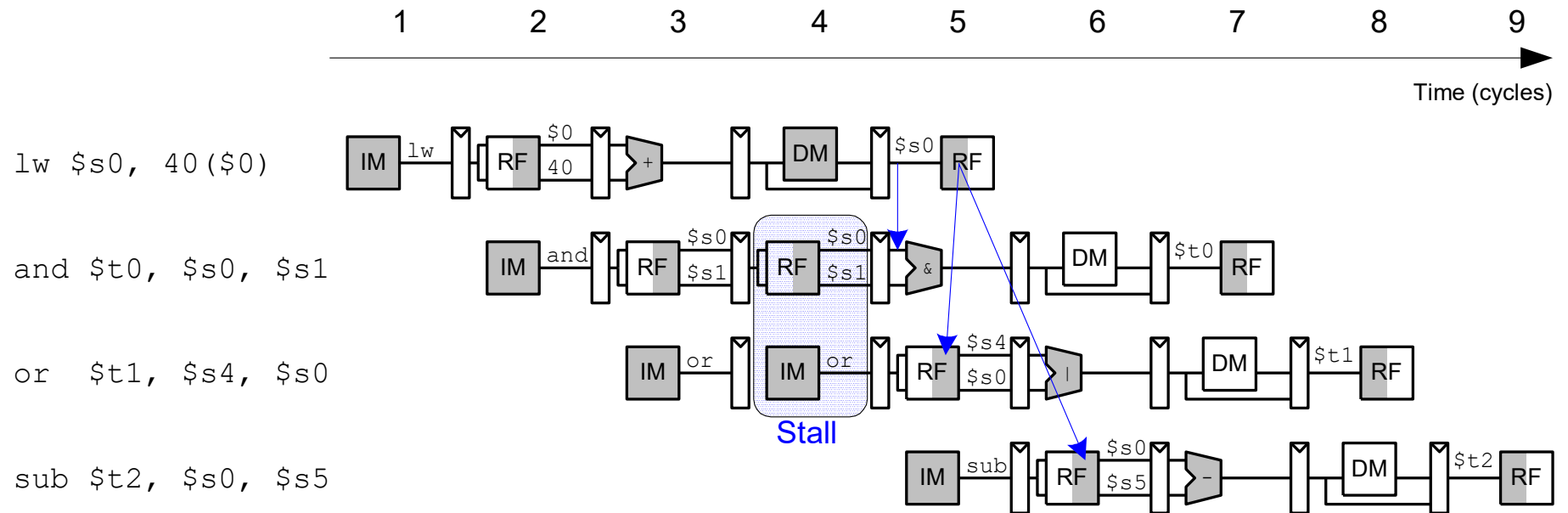
Data Forwarding



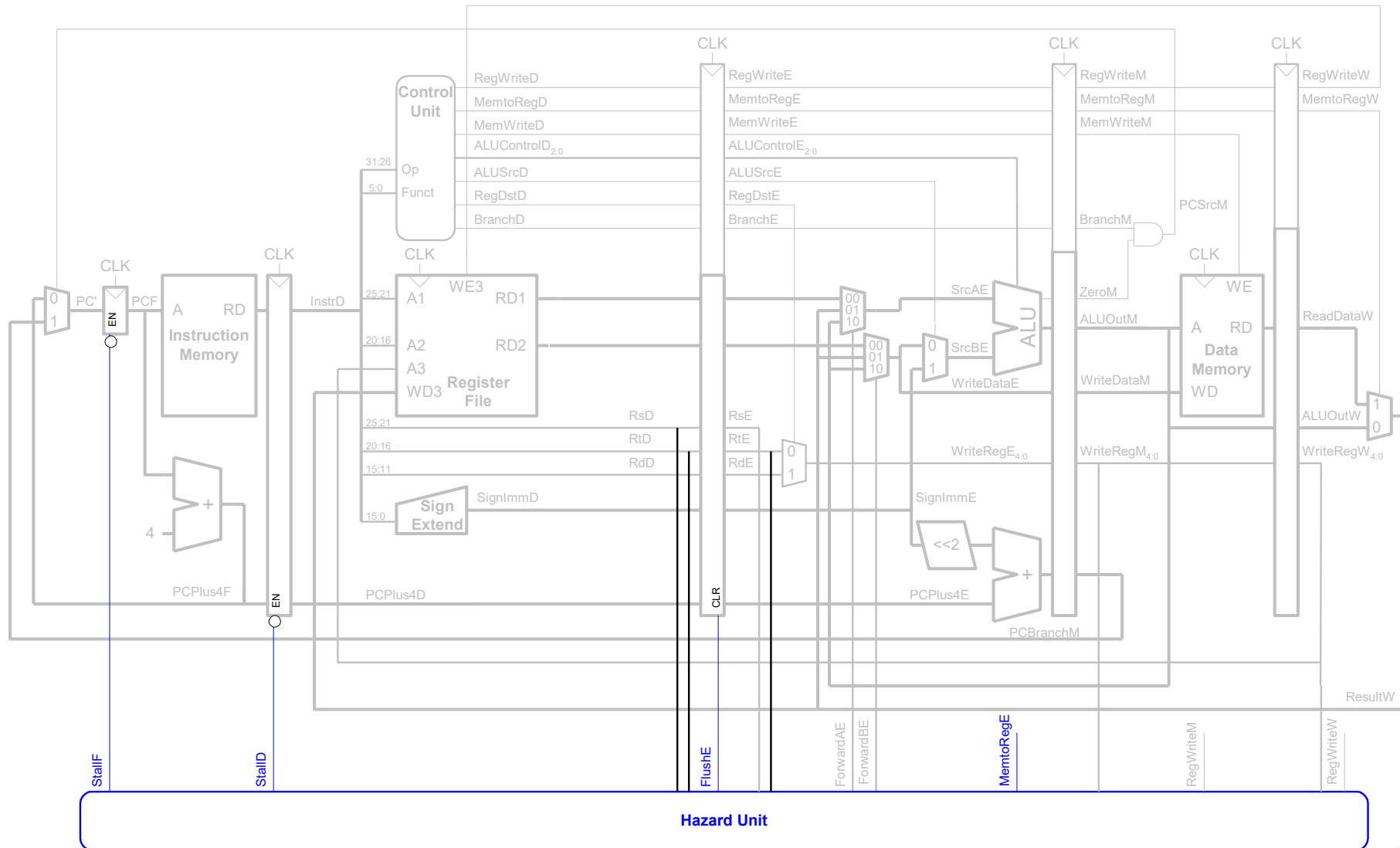
Stalling



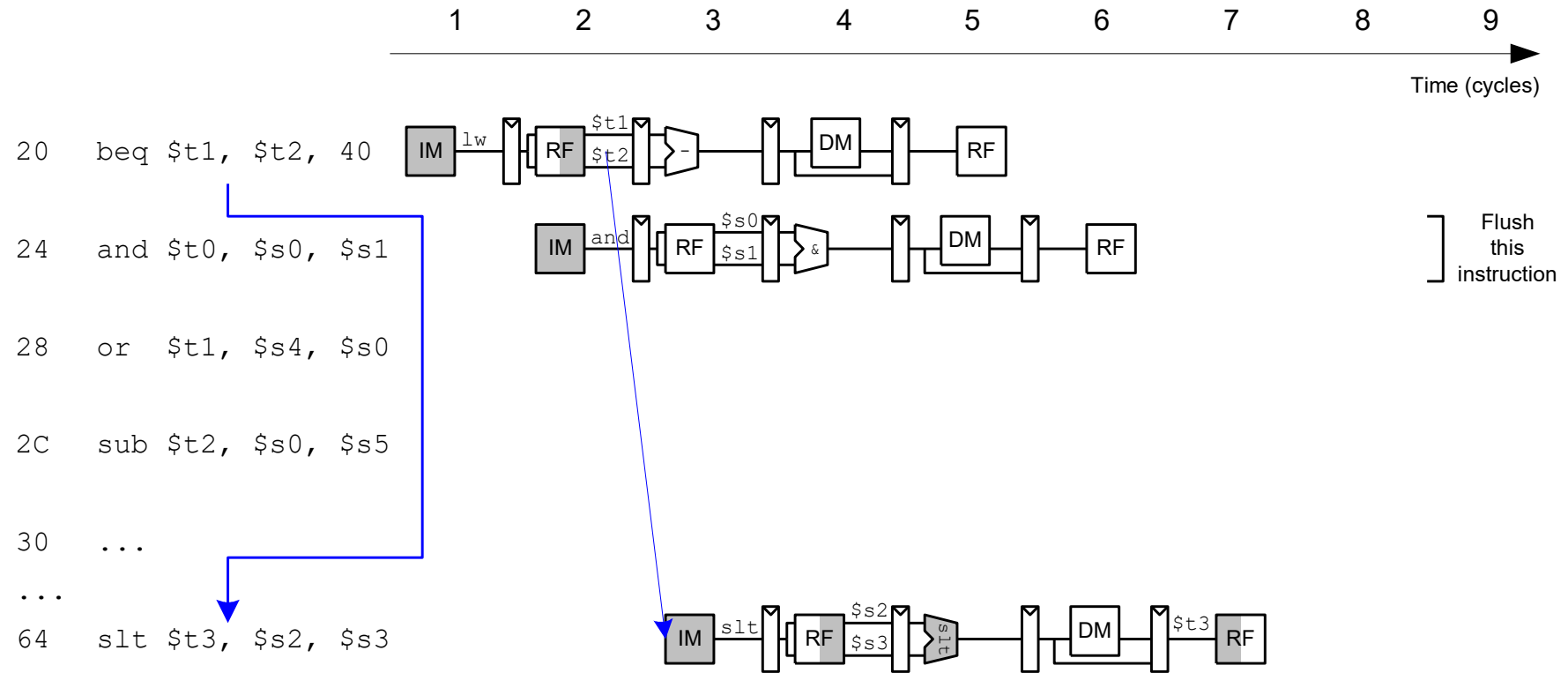
Stalling



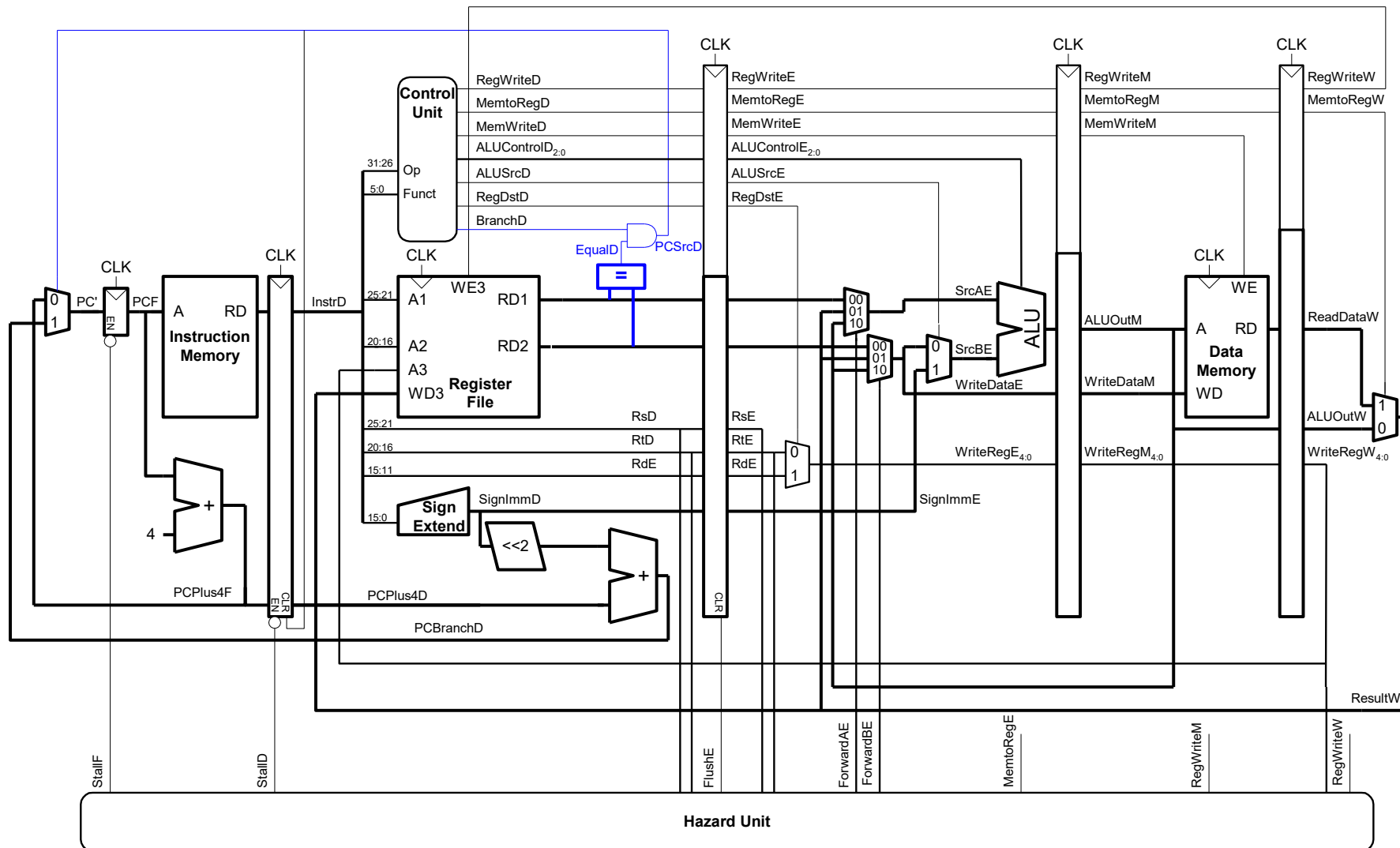
Stalling Hardware



Early Branch Resolution



Early Branch Resolution



Introduced another data hazard in Decode stage

Branch Prediction

- Guess whether branch will be taken
 - Backward branches are usually taken (loops)
 - Consider history to improve guess
- Good prediction reduces fraction of branches requiring a flush

Pipelined Performance Example

- SPECINT2000 benchmark:
 - 25% loads
 - 10% stores
 - 11% branches
 - 2% jumps
 - 52% R-type
- Suppose:
 - 40% of loads used by next instruction
 - 25% of branches mispredicted
 - All jumps flush next instruction
- **What is the average CPI?**
 - Load/Branch CPI = 1 when no stalling, 2 when stalling
 - $CPI_{lw} = 1(0.6) + 2(0.4) = 1.4$
 - $CPI_{beq} = 1(0.75) + 2(0.25) = 1.25$

$$\begin{aligned}\text{Average CPI} &= (0.25)(1.4) + (0.1)(1) + (0.11)(1.25) + (0.02)(2) + (0.52)(1) \\ &= 1.15\end{aligned}$$

Pipelined Performance

- Pipelined processor critical path:

$$T_c = \max \{$$

$$t_{pcq} + t_{mem} + t_{setup},$$

$$2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup}),$$

$$t_{pcq} + t_{mux} + t_{mux} + t_{ALU} + t_{setup},$$

$$t_{pcq} + t_{memwrite} + t_{setup},$$

$$2(t_{pcq} + t_{mux} + t_{RFwrite}) \}$$

takes 2 clock cycles to read.

Pipelined Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	$t_{pcq\ PC}$	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20
Equality comparator	t_{eq}	40
AND gate	t_{AND}	15
Memory write	$t_{memwrite}$	220
Register file write	$t_{RFwrite}$	100

$$\begin{aligned}
 T_c &= 2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup}) \\
 &= 2[150 + 25 + 40 + 15 + 25 + 20] \text{ ps} = \mathbf{550 \text{ ps}}
 \end{aligned}$$

Pipelined Performance Example

Program with 100 billion instructions

$$\begin{aligned}\textbf{Execution Time} &= (\# \text{ instructions}) \times \text{CPI} \times T_c \\ &= (100 \times 10^9)(1.15)(550 \times 10^{-12}) \\ &= \textbf{63 seconds}\end{aligned}$$

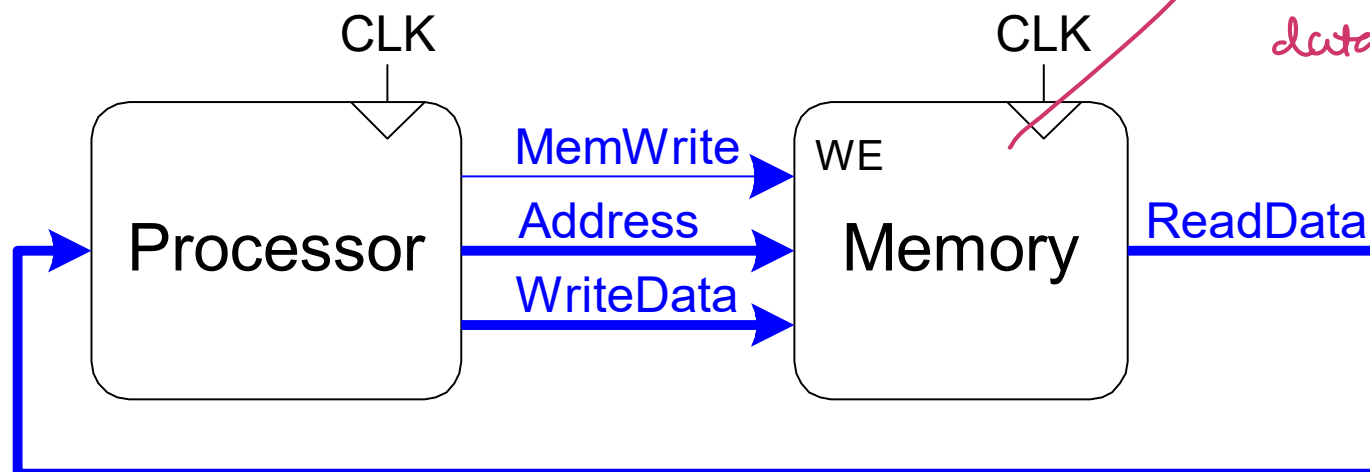
Processor Performance Comparison

Processor	Execution Time (seconds)	Speedup (single-cycle as baseline)
Single-cycle	92.5	1
Pipelined	63	1.47

Memory

- von Neumann architecture: stored-program model, one memory holds both program instructions and data
- Computer performance depends on:
 - Processor performance
 - Memory system performance

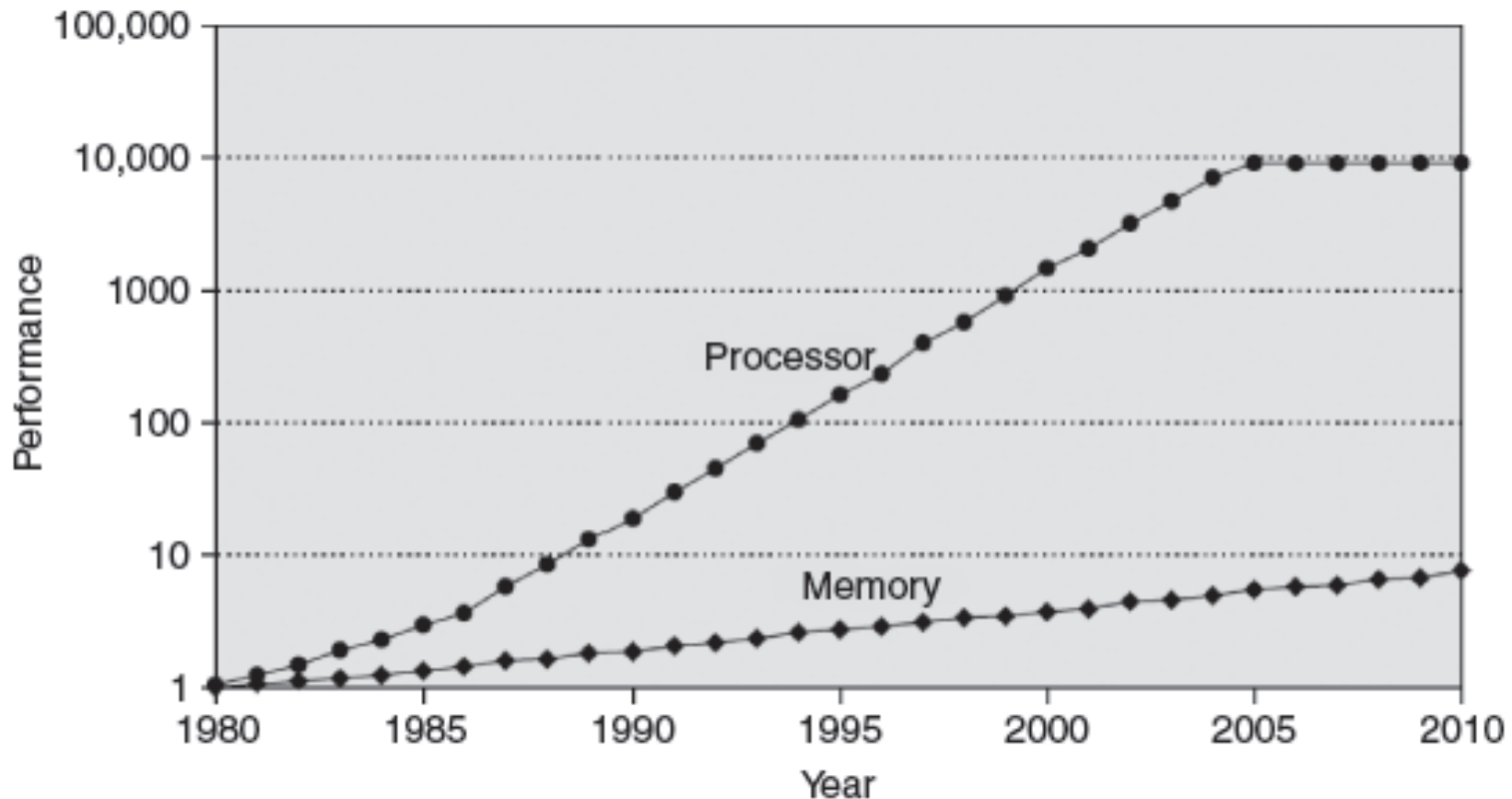
Memory Interface



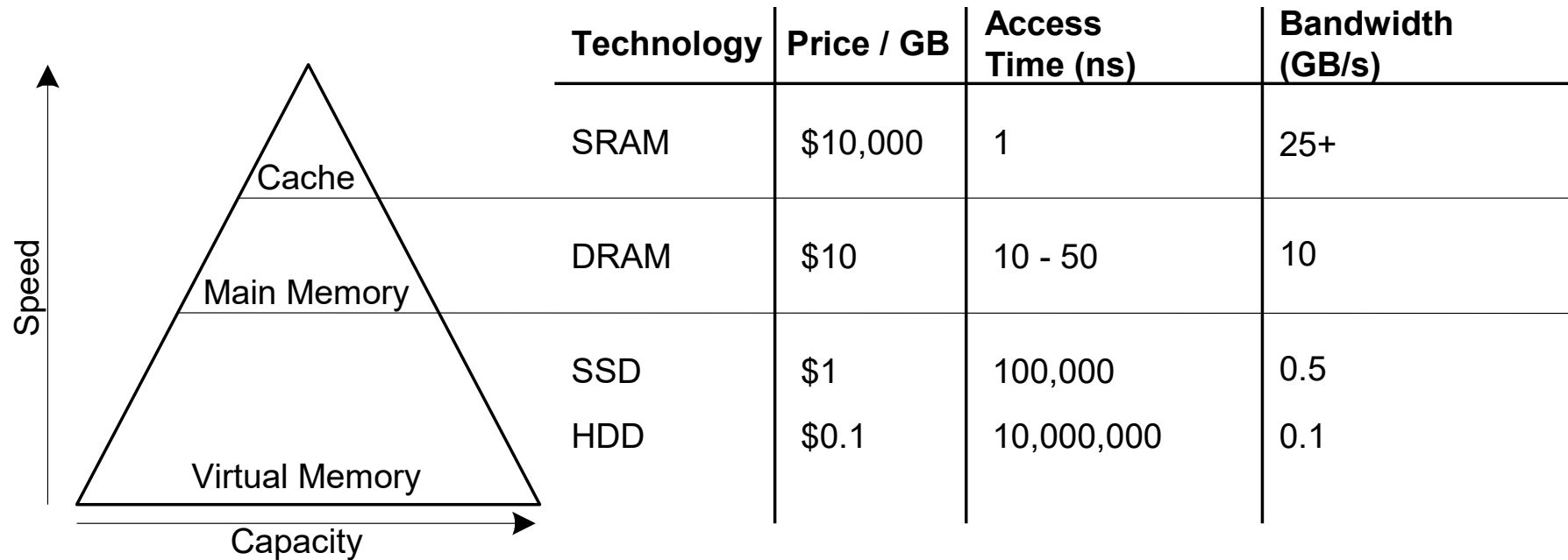
*contains both
instructions &
data memory*

Processor-Memory Gap

In prior chapters, assumed access memory in 1 clock cycle – but hasn't been true since the 1980's



Memory Hierarchy



Locality

Exploit locality to make memory accesses fast

- **Temporal Locality:**

- Locality in time
- If data used recently, likely to use it again soon
- **How to exploit:** keep recently accessed data in higher levels of memory hierarchy

*go back to example.
the for-loop*

- **Spatial Locality:**

- Locality in space
- If data used recently, likely to use nearby data soon
- **How to exploit:** when access data, bring nearby data into higher levels of memory hierarchy too

*more efficient than
for (i...) for (j...) acc[i][j] += 1
acc[j][i] += 1*

*parallel
computing
performance
analysis*

Memory Performance

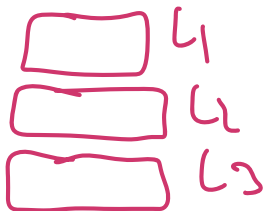
- **Hit:** data found in that level of memory hierarchy
- **Miss:** data not found (must go to next level)

Hit Rate $= \# \text{ hits} / \# \text{ memory accesses}$
 $= 1 - \text{Miss Rate}$

Miss Rate $= \# \text{ misses} / \# \text{ memory accesses}$
 $= 1 - \text{Hit Rate}$

- **Average memory access time (AMAT):** average time for processor to access data

$$\text{AMAT} = t_{L1} + MR_{L1}[t_{L2} + MR_{L2}(t_{L3} + \dots)]$$



$$= t_{L1} + MR_{L1} (t_{L2} + MR_{L2} t_{L3})$$

Memory Performance Example 1

- A program has 2,000 loads and stores
- 1,250 of these data values in cache
- Rest supplied by other levels of memory
hierarchy
- **What are the hit and miss rates for the cache?**

Memory Performance Example 1

- A program has 2,000 loads and stores
- 1,250 of these data values in cache
- Rest supplied by other levels of memory hierarchy
- **What are the hit and miss rates for the cache?**

$$\text{Hit Rate} = 1250/2000 = \mathbf{0.625}$$

$$\text{Miss Rate} = 750/2000 = \mathbf{0.375} = 1 - \text{Hit Rate}$$

Memory Performance Example 2

- Suppose processor has 2 levels of hierarchy: cache and main memory
- $t_{\text{cache}} = 1$ cycle, $t_{MM} = 100$ cycles
- **What is the AMAT of the program from Example 1?**

Memory Performance Example 2

- Suppose processor has 2 levels of hierarchy: cache and main memory
- $t_{\text{cache}} = 1$ cycle, $t_{MM} = 100$ cycles
- **What is the AMAT of the program from Example 1?**

$$\begin{aligned}\text{AMAT} &= t_{\text{cache}} + MR_{\text{cache}}(t_{MM}) \\ &= [1 + 0.375(100)] \text{ cycles} \\ &= \mathbf{38.5 \text{ cycles}}\end{aligned}$$